



UNIVERSIDAD NACIONAL DE COSTA RICA

Sede Regional Brunca | Escuela de Informática

EIF207 – Estructuras de Datos | II Ciclo 2026

INFORME TÉCNICO: PROYECTO I (15%)

Sistema Tetris

Estudiante:

Joseph Emmanuel Rodríguez Solano

119660751

Entorno de Desarrollo:

ZinjaI / MinGW GCC 6.3.0

1. Índice y Objetivos del Proyecto

1.1 Índice de Contenidos

Sección 1: Índice y Objetivos del Proyecto
Sección 2: Arquitectura y Desacoplamiento del Sistema
Sección 3: Tabla de Análisis Asintótico Completo
Sección 4: Máquina de Estados Finitos y Ciclo de Vida
Sección 5: Gestión Determinista de Memoria (Patrón RAII)
Sección 6: Referencias Bibliográficas (Formato APA)

1.2 Objetivo General

Diseñar, implementar y evaluar un sistema de juego Tetris en lenguaje C++ desacoplado de la capa gráfica SFML, fundamentado en la construcción manual de estructuras de datos lineales dinámicas bajo el patrón de gestión de memoria RAII (*Resource Acquisition Is Initialization*), integrando el análisis formal de complejidad asintótica ($\mathcal{O}$, Ω , Θ) y la evaluación empírica comparativa entre algoritmos de ordenamiento (Insertion Sort frente a QuickSort).

1.3 Objetivos Específicos

- Implementar manualmente, sin utilizar contenedores de la biblioteca estándar (STL), las estructuras requeridas por la lógica del juego: Cola FIFO para la secuencia de piezas (sistema de bolsas aleatorias *7-Bag*), Pila LIFO de capacidad unitaria para la reserva (*Hold*), Lista Doblemente Enlazada para el historial de movimientos y reproducción (*Replay* con funciones de deshacer y rehacer), Lista Enlazada de Filas para el tablero, y Cola de Prioridad Cronológica para la programación de eventos temporales.
- Encapsular la asignación dinámica en el *heap* y la desasignación determinista mediante destructores explícitos con `delete` en cada estructura propia, asegurando la prevención total de fugas de memoria (*memory leaks*).
- Determinar analíticamente las cotas teóricas de complejidad computacional (cota superior $\mathcal{O}$, inferior Ω y ajustada Θ) en el mejor, promedio y peor caso para las 16 operaciones algorítmicas fundamentales del sistema.

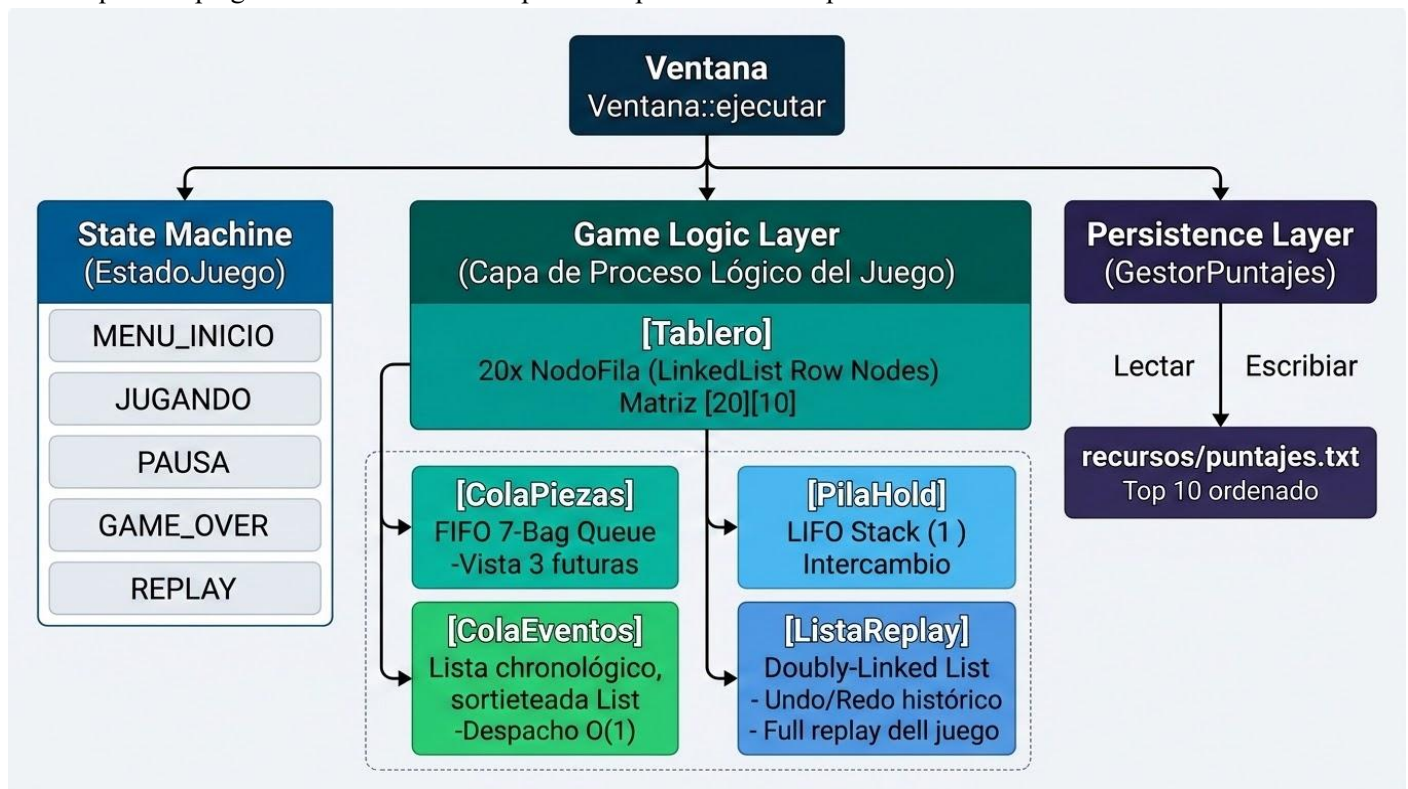
2. Arquitectura y Desacoplamiento del Sistema

El sistema implementa una arquitectura modular estratificada en dos dominios strictly independientes: la Capa de Presentación Gráfica e Interacción (SFML 2.5+) y el Núcleo de Datos y Cinemática (C++ puro sobre memoria dinámica).

La clase orquestadora Ventana centraliza el ciclo de vida del juego (Game Loop). Actúa como un mediador de control: captura las interrupciones del hardware y las convierte en instrucciones cinemáticas que delega a las estructuras dinámicas.

Para impedir el acoplamiento directo y evitar dependencias cruzadas, la comunicación entre la vista y las estructuras lineales se realiza exclusivamente mediante el paso de tipos primitivos (int, float, char, bool):

- La lógica de datos nunca incluye cabeceras de SFML (<SFML/Graphics.hpp>) ni almacena referencias a sf::Sprite, sf::Texture o sf::Drawable.
- Las colisiones, transiciones temporales y actualizaciones de estado se computan en matrices y nodos crudos antes de que la capa gráfica solicite los datos para su representación en pantalla.



2.1 Tablero de Juego (Tablero.h / Tablero.cpp)

El tablero modela la grilla de 10 columnas por 20 filas. A diferencia de una implementación sobre una matriz plana tradicional (int[20][10]), se estructura como una lista enlazada dinámica de 20 nodos (NodoFila). Cada nodo encapsula internamente un arreglo fijo de 10 celdas enteras.

- **Detección de Colisiones (colisiona)**: Toma las coordenadas cinemáticas (int x, int y, int rotacion) de una instancia transitoria de Pieza. Recorre la submatriz local de 4 x 4 del tetrominó y la contrasta contra los valores almacenados en los nodos de fila pertinentes sin iterar celdas innecesarias.
- **Fijación de Piezas (fijarPieza)**: Cuando el movimiento vertical es bloqueado por colisión inferior, las celdas activas del tetrominó se escriben de manera permanente en los arreglos de hasta 4 nodos de fila consecutivos.
- **Limpieza de Líneas y Gravedad en Tiempo Constante**:
 - 1. El método detectarFilasLlenas recorre los 20 nodos. Si las 10 columnas de un nodo tienen un valor distinto de cero, el nodo se marca para extracción.
 - 2. eliminarFilaCompleta desvincula el nodo de la lista (anterior->sig = actual->sig) y destruye su bloque de memoria dinámicamente mediante delete actual en tiempo constante O(1).

- 3. insertarFilaVacíaCabeza instancia un nuevo NodoFila en blanco y lo acopla al inicio de la lista (nuevo->sig = cabeza; cabeza = nuevo;) en $O(1)$.

Este diseño convierte la caída física de bloques tras limpiar líneas en una simple reestructuración de punteros, eliminando el costo de copiar arreglos contiguos en memoria.

2.2 Secuencia de Piezas Futuras (ColaPiezas.h / ColaPiezas.cpp)

Administra el flujo de tetrominós mediante una Cola FIFO dinámica enlazada manualmente mediante punteros frente y fin.

- Mecánica 7-Bag (Bolsa de 7): Para garantizar una distribución justa y sin sequías de piezas, implementa el algoritmo de barajado Fisher-Yates sobre un arreglo con los 7 tipos de tetrominós clásicos (I, O, T, S, Z, J, L). Los 7 elementos resultantes se enlazan consecutivamente en la cola al agotarse la bolsa previa.
- Extracción y Despacho: La pieza en juego se extrae de la cabeza (frente) en $O(1)$, advancing el puntero frontal y liberando el nodo despachado con delete.
- Previsualización No Destructiva (consultarPorIndice): El panel visual de la interfaz requiere mostrar las 3 piezas siguientes. Mediante una navegación temporal de punteros acotada a k in $\{0, 1, 2\}$, la cola retorna los identificadores correspondientes sin alterar el orden ni desencolar nodos.

2.3 Casilla de Espera (PilaHold.h / PilaHold.cpp)

Modela la funcionalidad táctica de retención de piezas mediante una Pila LIFO dinámica con capacidad unitaria fija (máximo un nodo en el puntero tope).

- Mecánica de Intercambio (Swap): Al pulsar la tecla C, el sistema evalúa el estado del contenedor. Si la pila está vacía, la pieza activa se transfiere al tope mediante asignación dinámica y se extrae el siguiente tetrominó de ColaPiezas. Si ya existe una pieza retenida, se realiza un intercambio atómico de identificadores entre la pieza activa y el nodo en tope en $O(1)$, bloqueando nuevos intercambios hasta que la pieza en juego se asiente en el tablero.

2.4 Sistema de Telemetría y Repetición (ListaReplay.h / ListaReplay.cpp)

Registra la evolución cronológica de la partida mediante una Lista Doblemente Enlazada. Cada nodo almacena una estructura liviana de datos primitivos que representa una captura atómica de estado: coordenadas del bloque (int posX, int posY), orientación (int rotacion) e identificador de pieza (int tipoPieza).

- Deshacer y Rehacer en Tiempo de Juego: Gracias a los punteros simétricos ant y sig, el usuario puede retroceder (Z) o avanzar (X) pasos interactivos durante la partida en tiempo estrictamente constante $O(1)$ (actual = actual->ant o actual = actual->sig), actualizando la cinemática sin necesidad de reiniciar la partida.
- Reproducción de Partida (Replay): Al alcanzar el estado GAME_OVER, el método irAlInicio() sitúa el puntero de control en el nodo primero. Desde allí, permite ejecutar una reproducción secuencial paso a paso o continua mediante un temporizador automático, reconstruyendo visualmente toda la partida desde el paso cero hasta el desenlace.

2.5 Programador Temporal de Eventos (ColaEventos.h / ColaEventos.cpp)

Gestiona los hitos dinámicos del juego en tiempo real (ajustes en la velocidad de caída por nivel, temporizadores de destello y efectos especiales). Opera como una cola con prioridad donde la prioridad está definida por una marca de tiempo absoluta en segundos (float tiempoDisparo).

- Mantenimiento del Invariante Cronológico (insertarOrdenado): No se insertan nodos al final para luego ordenarlos. Todo nuevo evento se inserta en su posición cronológica exacta. Si su tiempo es inferior al de la cabeza actual, se posiciona en el frente inmediatamente ($\Omega(1)$); en caso contrario, se realiza un recorrido secuencial hasta intercalarlo entre dos nodos donde $\text{actual->tiempoDisparo} \leq T_{\text{nuevo}} < \text{actual->sig->tiempoDisparo}$ ($O(E)$).
- Despacho Inmediato en el Bucle Principal: Dado que la cabeza de la cola conserva por diseño el evento con el vencimiento más inminente, el ciclo de juego solo evalúa en cada fotograma si el reloj del sistema alcanzó la marca frontal ($\text{reloj.asSeconds()} \geq \text{frente->tiempoDisparo}$). Al cumplirse, despacha el efecto y desacopla el nodo frontal en tiempo constante $O(1)$ senza iterar la colección.

2.6 Persistencia y Clasificación de Récords (GestorPuntajes.h / GestorPuntajes.cpp)

Administra el almacenamiento y clasificación del Top 10 de mejores puntajes en el archivo plano recursos/puntajes.txt.

- Algoritmos Propios de Ordenamiento: Para dar cumplimiento a las restricciones técnicas sin recurrir a `std::sort`, el módulo contiene dos algoritmos manuales:
 - 1. Insertion Sort: Algoritmo de complejidad cuadrática $O(N^2)$ que desplaza progresivamente los registros para insertar cada elemento en su sublista ordenada.
 - 2. QuickSort: Algoritmo de división y conquista con cota promedio $O(N \log N)$ basado en la partición recursiva del arreglo alrededor de un valor pivote.
- Selección Dinámica: Mediante la tecla T, el jugador puede alternar cuál de los dos métodos se ejecuta para ordenar la tabla de puntajes antes de su renderizado en pantalla o persistencia en disco.

3. Tabla de Análisis Asintótico Completo

A continuación se presenta la caracterización formal de complejidad computacional para las 16 operaciones clave del sistema Tetris:

Estructura / Operación	Mejor (Ω)	Prom. (Θ)	Peor (O)	Justificación Algorítmica Concisa
1. ColaPiezas::encolarBolsa (7-Bag)	$\Omega(1)$	$\Theta(1)$	$O(1)$	Generación y encolado fijo de 7 elementos mediante permutación aleatoria Fisher-Yates en tiempo constante.
2. ColaPiezas::desencolar	$\Omega(1)$	$\Theta(1)$	$O(1)$	Remoción del nodo en la cabeza de la cola mediante punteros directos frente/final.
3. ColaPiezas::consultarPorIndice(k)	$\Omega(1)$	$\Theta(k)$	$O(1)$	Acceso acotado a las próximas piezas donde $k \leq 2$; se recorren a lo sumo 2 nodos en tiempo acotado $O(1)$.
4. PilaHold::push / pop / top	$\Omega(1)$	$\Theta(1)$	$O(1)$	Operaciones LIFO en el tope de la pila mediante modificación inmediata de puntero de cabeza.
5. Tablero::colisiona	$\Omega(1)$	$\Theta(1)$	$O(1)$	Verificación de superposición evaluando una matriz fija de 4×4 celdas sobre la cuadrícula lógica.
6. Tablero::fijarPieza	$\Omega(1)$	$\Theta(1)$	$O(1)$	Transferencia de 4 bloques a la estructura del tablero en un número constante de iteraciones.
7. Tablero::detectarFilasLlenas	$\Omega(1)$	$\Theta(1)$	$O(1)$	Inspección de un tablero de dimensión fija ($20 \times 10 = 200$ celdas), requiriendo exactamente 200 comparaciones.
8. Tablero::eliminarFilaCompleta	$\Omega(1)$	$\Theta(1)$	$O(1)$	Desconexión de un nodo fila en la lista enlazada mediante ajuste directo de punteros previo/siguiente.
9. Tablero::insertarFilaVacíaCabeza	$\Omega(1)$	$\Theta(1)$	$O(1)$	Asignación de un nuevo nodo fila e inserción en la cabeza de la lista del tablero en $O(1)$.
10. ListaReplay::registrarEstado	$\Omega(1)$	$\Theta(1)$	$O(1)$	Inserción al final de la lista doblemente enlazada almacenando el struct ligero del estado del movimiento.
11. ListaReplay::deshacerPaso (Z) / rehacerPaso (X)	$\Omega(1)$	$\Theta(1)$	$O(1)$	Desplazamiento del puntero actual hacia atrás o adelante mediante punteros prev y next.

Estructura / Operación	Mejor (Ω)	Prom. (Θ)	Peor (O)	Justificación Algorítmica Concisa
12. ListaReplay::reproducirCompleta	$\Omega(M)$	$\Theta(M)$	$O(M)$	Recorrido secuencial desde el inicio hasta el final de la lista para M estados grabados.
13. ColaEventos::insertarOrdenado	$\Omega(1)$	$\Theta(E)$	$O(E)$	Búsqueda lineal e inserción ordenada por timestamp de ejecución sobre una lista de E eventos activos.
14. ColaEventos::extraerFrente	$\Omega(1)$	$\Theta(1)$	$O(1)$	Extracción del evento listo para ejecución ubicado en la cabeza de la cola prioritaria temporal.
15. GestorPuntajes::ordenarInsercion	$\Omega(N)$	$\Theta(N^2)$	$O(N^2)$	Insertion Sort sobre N elementos; $\Omega(N)$ con arreglo ya ordenado, $\Theta(N^2)$ y $O(N^2)$ en el caso promedio/peor.
16. GestorPuntajes::ordenarQuickSort	$\Omega(N \log N)$	$\Theta(N \log N)$	$O(N^2)$	QuickSort con pivote estándar; $\Omega(N \log N)$ y $\Theta(N \log N)$ promedio, $O(N^2)$ con peor elección sistemática de pivote.

4. Máquina de Estados Finitos y Ciclo de Vida (EstadoJuego)

El controlador Ventana gobierna el flujo de interacción a través de una máquina de estados finitos modelada mediante el enumerador fuertemente tipado EstadoJuego:

- 1. MENU_INICIO: Presenta la pantalla de bienvenida y valida la entrada alfanumérica del nombre del jugador mediante teclado antes de inicializar las estructuras dinámicas del tablero.
- 2. JUGANDO: Estado dinámico principal. Procesa el descenso de las piezas, comprueba colisiones, atiende las solicitudes de intercambio (hold), actualiza la traza en ListaReplay y evalúa los disparos de temporizadores en ColaEventos. Al limpiar una o más filas, detiene momentáneamente la caída activa para proyectar una animación de destello (200 ms) antes de desacoplar los nodos de fila.
- 3. PAUSA: Se activa con la tecla Escape. Suspende la marcha de los relojes de juego de SFML y despliega un menú modal interactivo con opciones navegables (Continuar, Reiniciar, Salir), impidiendo el movimiento o caída de piezas.
- 4. GAME_OVER: Se dispara cuando un tetrominó recién generado colisiona en su posición inicial de entrada. Verifica si la puntuación obtenida ingresa al Top 10 histórico, ordena el registro con el algoritmo activo y persiste los datos en disco.
- 5. REPLAY: Habilita el control interactivo del historial de la partida. Mediante ListaReplay::irAlInicio(), inicializa la visualización en el primer movimiento. Permite avanzar y retroceder paso a paso con las flechas direccionales, iniciar la reproducción automática con la barra espaciadora, reiniciar al paso inicial (I) o alternar la visibilidad de las estadísticas en pantalla (H) para observar la cuadrícula despejada.

5. Gestión Determinista de Memoria (Patrón RAII)

Para garantizar la estabilidad del sistema y asegurar la ausencia total de fugas de memoria (memory leaks), todas las estructuras dinámicas encapsulan la reserva y destrucción de recursos bajo el patrón RAII (Resource Acquisition Is Initialization):

- `Tablero::~~Tablero()`: Recorre secuencialmente la lista de 20 filas dinámicas. Antes de desasignar el nodo actual con `delete` actual, almacena un puntero temporal a `actual->sig`, garantizando la liberación limpia de la lista enlazada completa.
- `ColaPiezas::~~ColaPiezas()` y `ColaEventos::~~ColaEventos()`: Implementan ciclos de vaciado continuo invocando sus rutinas internas de extracción frontal hasta que los punteros frente y fin quedan apuntando a `nullptr` y el conteo de elementos se reduce a cero.
- `PilaHold::~~PilaHold()`: Evalúa el puntero tope; si existe una pieza retenida en memoria dinámica, ejecuta su liberación inmediata con `delete`.

`ListaReplay::~~ListaReplay()`: Realiza un barrido bidireccional desde la cabeza (primero), destruyendo individualmente cada nodo de estado mediante `delete` para evitar referencias huérfanas al cerrar la aplicación o reiniciar una partida.

6. Referencias Bibliográficas (Formato APA)

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

Drozdek, A. (2013). *Data Structures and Algorithms in C++* (4th ed.). Cengage Learning.

Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.

Weiss, M. A. (2014). *Data Structures and Algorithm Analysis in C++* (4th ed.). Pearson.